ELLY Business Development Portal — Comprehensive Audit

Date: 2026-10-31

Branch audited: `worktree-report-agent` (11 commits ahead of `main`; merge-base `76d4f6d`)

Method: Six parallel read-only subagent audits (Frontend, AI/Copilot, Integrations & Automation, CRM Core Entities, Auth/Roles/CEO Dashboard/Security, Delta-check vs prior audit), cross-referenced against source. No code was modified, no database rows or secret values were read or printed. This report supersedes `PROJECT_AUDIT_2026-10-28.pdf` for every section it re-covers; sections not re-examined are carried forward and marked as such.

Constraints honored: No code changes made. No feature assumed working from UI presence alone — every claim below is backed by a specific file:line citation. Frontend-to-backend wiring and actual database operations were verified by reading both sides of each call. AI workflows and external integrations were traced end-to-end, not assumed from docstrings. Working functionality is explicitly distinguished from mock/template/placeholder behavior throughout.

---

0. Executive Summary

The system is substantially real: no stub pages, no mock-data arrays, nearly all claimed CRUD operations hit a live SQLite database through SQLAlchemy, and the Copilot's data-returning tools are genuinely wired to real queries. The architecture is sound for a 3-4-person tool. But the audit surfaced a materially larger set of correctness and security defects than the 2026-10-28 audit found, concentrated in four areas:

- **The pipeline/stage system is internally inconsistent and self-corrupting.** Two incompatible stage vocabularies coexist, and an auto-sync job silently reverts organizations to earlier stages every time the Connections board is opened.
- **Several "send" and "update" actions report success without doing the real-world action they claim.** The Copilot's `send_email` writes a DB row and claims "dispatched" without sending mail; `update_organization_status` is LLM-callable but has no backing handler; campaign sequence steps never auto-advance despite being fully modeled.
- **Executive/team-performance reporting is numerically wrong for most of its metrics** due to a type mismatch (comparing user IDs to name strings), which will be directly visible in the newly-shipped executive report feature and the CEO-facing Analytics page.
- **Access control gaps:** several BD-only entity types (Customers, Demos, Proposals, Negotiations, Tasks, Calls) are reachable by any authenticated user regardless of role; audit logging is inconsistent across entities; and a handful of endpoints have real security exposure (open redirect, unauthenticated geocoding proxy, forgeable tracking counters).

Of the 2026-10-28 audit's 14 action items, **7 are now fixed, 7 remain open** (see Section 11). Of the 6 original HIGH items, 2 are closed (duplicate .env keys, rate-limiter risk now documented) and 4 remain open (pytest error rate unverified, no Alembic, no CI, JWT in localStorage).

---

1. Frontend — Navigation Modules

Correction to prior documentation: `Navbar.tsx:111` contains a stale comment claiming "17-item" navigation. The actual count is **23 nav items for all users, 25 for ADMIN** (adds Settings + Admin Console). All 23/25 routes resolve to real components — no stub pages, no hardcoded mock-data arrays were found anywhere in the nav tree. `npx tsc --noEmit` passes clean.

Module	Status	Notes
Organizations, Contacts, Opportunities, Meetings, Partnerships	Implemented	Full list/create/edit/detail, real backend CRUD
Communications	Implemented (one dead sub-feature)	Timeline aggregation endpoint is broken server-side (see §4)
Demos, Proposals, Negotiations, Customers	**Partial/Broken**	List + create only in UI; no update/delete wired despite backend supporting it; cards not clickable; every create hardcodes `organization_id: 1` with no org picker
Calendar	**Partial**	Event click handler is a `console.log` stub — no detail view, no edit
Notifications	**Partial**	Bell dropdown renders off-screen on mobile widths
Calls	**Broken (orphaned)**	Has a working route but no nav entry; also hardcodes `organization_id: 1`
Integrations	Dead code	`services/integrations.ts` exists but is imported nowhere in the frontend
Follow-ups	**Partial**	No dedicated page/route; only reachable via a modal and detail-page widgets; `/grouped`, PATCH, cancel, delete never called from UI

Frontend bugs found (backend routes all verified to exist and respond correctly where noted):

- Hardcoded `organization_id: 1` in 5 views (Demos, Proposals, Negotiations, Customers, Calls) — every record created from these views attaches to org #1 regardless of which org the user intended.
- Silent create-failure: errors only go to `console.error`, no user-facing toast/message, across multiple create forms.
- Mobile notification dropdown renders off-screen/invisible (CSS positioning bug).
- `discovery.ts:187` calls `GET` where the backend route is `POST` — currently an unused method path (latent, not yet triggered).
- Layout breakpoint mismatch: sidebar appears at the `lg` breakpoint but main-content padding assumes the sidebar from `md`, producing a visibly broken layout band between 768px and 1023px.
- Calendar event click is a no-op stub.
- Calls page has no nav entry (orphaned).

- Double nav active-highlight: `startsWith` matching causes two nav items to show "active" simultaneously on some routes.
- Stale "17-item" comment in `Navbar.tsx:111`.
- ~30 `alert()` calls used instead of the app's toast pattern — inconsistent UX, and `alert()` blocks the JS thread.

---

2. Backend Architecture

Carried forward from the 2026-10-28 audit (not fully re-walked this pass, confirmed still boots): FastAPI + SQLAlchemy + SQLite, 282 registered routes (`python -c "from app.main import app; print(len(app.routes))"` → `282`, no import errors), no service layer for most CRM entities (routers query the DB directly — see §4), APScheduler running 6 background jobs (not 5, as the prior audit stated — see §8), no Alembic, no CI.

---

3. AI System (Copilot / RAG)

Architecture confirmed working: two-tier intent routing — a regex fast-path (`intent_router.py`, ~30 patterns, 0 tokens, instant) and an LLM function-calling fallback for anything the regex misses, verified via live trace. The knowledge-base search tool is named `search_knowledge_base`, not `kb_search` as previously assumed.

Critical correctness/trust findings:

- `update_organization_status` **is exposed to the LLM as a callable tool but has no backing handler.** Every invocation returns an error. The Copilot can be asked to do this and will fail every time.
- `send_funding_outreach` **bypasses the Copilot's own anti-autonomous-send policy.** It is LLM-callable, is described in its own schema as sending "immediately with NO approval step," and calls `CampaignService.quick_send`, which auto-approves and sends — directly contradicting the confirm-before-send guardrail the rest of the Copilot enforces.
- `send_email` **does not send email.** It writes a `Communication` row and an audit event, then replies "Successfully dispatched" to the user — a materially misleading success message. It also likely reads `user.name` where the model attribute is `full_name` (latent AttributeError risk).
- `draft_outreach_email` **is a pure template, not an LLM draft, and hardcodes the wrong brand name** ("Electra" instead of "ELLY").
- **No real workflow engine exists.** `TaskType.COMPLEX_WORKFLOW` routes to `"complex_tool"`, not a `"workflow"` handler; multi-step requests are handled only by the LLM's own tool-call loop (capped at 10 rounds), which is not the same as orchestrated workflow execution.
- **The regex fast-path can silently drop steps from multi-part requests.** E.g., "find hospitals and draft emails" routes to `discover_web` only — the draft step is dropped with no indication to the user. Discovery itself writes directly to the CRM with no approval step, hardcodes `potential_fit_score=65` for every discovered org, and can misattribute scraped emails to the wrong contact.
- **RAG / embeddings risk:** `text-embedding-004` may be retired by Google (needs live verification); if so, the system silently falls back to plain Jaccard keyword matching with no warning to the user that semantic search degraded. KB upload is declared `async def` but performs synchronous per-chunk HTTP calls, blocking the event loop on large files. Only UTF-8 text/markdown is accepted — no PDF/DOCX ingestion despite this being a stated use case.
- **Stale Gemini model IDs** (`gemini-2.0-flash`, fallback `gemini-1.5-flash`) are both retired/404 in production (confirmed via direct API call this session, independently corroborated by this subagent). A working

replacement (`gemini-3.5-flash-lite`) was identified and verified live but not yet patched into `llm_router.py:58-59` — **awaiting user decision**, not yet applied.

- **The current user message is sent to the LLM twice** due to a duplicate-append bug (`engine.py:180-186`, `1098-1103`, `1135`), inflating token usage on every LLM-routed turn. Chat history is also unbounded with no truncation — long-running conversations will eventually exceed context limits or blow up cost.
- `auto_generate_followups` **mislabels every PENDING follow-up as "overdue"** regardless of its actual due date.
- **Six registry tools are defined but unreachable by the LLM** because they're regex-only, never exposed to the function-calling schema: `get_stage_organizations`, `search_partnerships`, `get_funnel_reports`, `search_customers`, `search_proposals`, and the `discover_organizations` alias.

---

4. CRM Core Entities

All 13 CRM routers are registered and query the database directly — there is no service layer except a few inline helpers (meeting scheduler, Sheets sync, audit). Full detail status per entity:

Entity	Status	Key issue
Organizations	Partial	Status/priority changes write no audit trail and accept any string value
Connections (pipeline/Kanban)	**Broken**	See Finding 1 below — the board actively corrupts data
Contacts	Implemented	One dead endpoint (`suggest-next-action`) throws `AttributeError`, unused by frontend
Opportunities	Implemented	—
Communications	Partial	`/timeline` aggregation endpoint throws a 500 under normal conditions (Finding 2)
Calls	Broken (frontend) / Partial (API)	Hardcoded org_id:1 in UI; can move an org's stage backward unconditionally
Meetings	Implemented	Same dead-endpoint pattern as Contacts
Demos, Proposals, Negotiations, Customers	**Broken**	All hardcode `organization_id: 1`; list+create only in UI; `/customers/convert` never called
Follow-ups	Partial	No dedicated page/route; UI never calls `/grouped`, PATCH, cancel, or delete
Partnerships	Implemented	—

Findings (verified against code, not inferred):

- **[CRITICAL] The Connections/Kanban board silently reverts organization stages on every page load.** `ConnectionStatus` defines a 10-stage funnel, but the board, `MoveStageInput`, and the frontend's `ConnectionStatus` type only recognize the legacy 7-stage vocabulary. `auto_sync_pipeline` treats any value it doesn't recognize as rank 0 and rewrites the organization's status whenever a computed target outranks it — and this function runs automatically on every `GET /connections/pipeline` call (auto-sync defaults to `True`). Net effect: simply opening the Connections board can move an organization already at "Customer" or "Negotiation" (if it has any meeting record) back down to "Meeting." Compounding this, `compute_target_stage` checks for opportunity-stage strings (`"Closed Won"`, `"Partnership Won"`, etc.) that the opportunity schema's own validator rejects, so the "Partnership" auto-sync rule can never fire correctly.
- **[HIGH] The relationship timeline endpoint returns HTTP 500 whenever a meeting or follow-up falls in the queried date range.** `communications.py:303,332` pass an integer `created_by`/`performed_by` into a schema field typed `Optional[str]`; Pydantic v2 rejects this with a validation error. This breaks the `RelationshipTimeline` component on the Organization, Contact, Opportunity, and Partnership detail pages. The existing test for this endpoint only covers an org with communications, not meetings/follow-ups, so it doesn't catch the bug.
- **[MEDIUM] Three independent, uncoordinated activity logs exist** (`ContactActivity`, `AuditLog`, `CampaignContactTimeline`), each covering a different, overlapping subset of entity mutations. No single source of truth for "what happened to this org."
- **[MEDIUM] Two duplicate code paths exist for the same actions**: logging a call (`/calls` vs. `/communications/quick-log/phone`, with different side effects), and converting an opportunity to a partnership (one endpoint used by the frontend, a second one that nothing calls).
- **[MEDIUM] Demos/Proposals/Negotiations/Customers/Calls use `get_current_user` instead of `require_business_development`,** inconsistent with every other CRM router, and their update/delete paths write no audit record at all.

---

5. Auth, Roles & Permissions

Five roles exist: `BUSINESS_DEVELOPMENT`, `HEALTHCARE`, `ADMIN`, `USER`, `DEV`. There is no CEO/manager role — CEO-level access is just `ADMIN`. `DEV` is new this week (added for the executive-report feature) and is **not** a superset or subset of `ADMIN`: the two gates don't overlap (see Finding 4 below).

Router group	Gate	Reachable by
Admin console (users, team, ownership, audit logs, settings)	`require_admin`	ADMIN only
Executive report generation/download	`require_role("DEV")`	DEV only — **ADMIN cannot use this feature**
Most BD routers (orgs, contacts, opportunities, meetings, campaigns, discovery, copilot, etc.)	`require_business_development`	ADMIN, BD, DEV
Customers, Demos, Proposals, Negotiations, Tasks, Calls, Daily Priorities, Notifications, Integrations ingest	`get_current_user` only	**Any authenticated user, including HEALTHCARE/USER**

Router group	Gate	Reachable by
Login, dropdowns, geocoding, OAuth callback, tracking pixels, inbound webhooks	Public	Anyone (OAuth callback and inbound webhooks are HMAC-protected; geocoding and tracking are not)

Findings:

- **[HIGH] HEALTHCARE and USER-role accounts can read and modify Customers, Demos, Proposals, Negotiations, Tasks, Calls, and Daily Priorities through the API**, even though the frontend blocks them from these pages. The API itself has no role check on these routers — only `get_current_user`.
- **[HIGH]** `GET /api/admin/users/{id}` **throws** `AttributeError` **and returns HTTP 500** — it filters on `Meeting.organizer_id`, which doesn't exist (the model's field is `owner_id`). `AdminUserDetailPage.tsx` calls this endpoint directly, so the admin user-detail page is broken for every user.
- **[MEDIUM] No ownership checks on mutations.** Any BD user can delete any organization, update or delete any other user's task, or reassign any opportunity's owner — with no owner/role check and, in the opportunity-owner case, no audit record either.
- **[MEDIUM] ADMIN and DEV gates don't overlap**, confirm if intentional: ADMIN (the de facto CEO) cannot generate or download the executive report; DEV cannot reach the admin console; admin headcount views exclude DEV users; any DEV can download any other DEV's generated report by guessing a sequential ID (no `created_by` check).
- **[MEDIUM]** `owner_scope=all` **is cosmetic, not enforced.** Non-admin users can pass this query parameter on organizations/contacts and see everything anyway — either enforce it server-side or remove the illusion of scoping.
- **[MEDIUM] The audit-log viewing endpoint is readable by any BD/DEV user**, including other users' login failures and admin role/status changes — this should be `require_admin` (an admin-only copy of the same endpoint already exists and is unused for this purpose).
- **[LOW] No server-side logout and no token revocation on password change.** Combined with JWT-in-localStorage (see §9), a stolen token remains valid until natural expiry.
- **[LOW] Frontend admin role pickers offer** `HEALTHCARE`**,** which the backend validator rejects outright (422) — and no admin UI offers `DEV` at all.

---

6. CEO Dashboard

Capability	Status
Org-wide KPIs, pipeline/conversion/meeting/opportunity-value metrics	Implemented
Opportunity tracking across the team, filterable by owner	Implemented
Employee activity monitoring	**Broken** — see Finding 1 (type mismatch zeroes out most per-rep metrics)
Team-wide task tracking	**Missing** — the task dashboard is strictly per-user; there is no team rollup anywhere

Capability	Status
Access control on org-wide analytics	Partial — any BD or DEV user (not just ADMIN) can see every rep's name, email, and individual performance numbers

Findings:

- **[CRITICAL] Team-performance KPIs are wrong for 4 of 6 metrics.** `report_agent_service.py` compares `created_by` (an Integer user-ID column on Organizations, Contacts, Meetings, and Follow-ups) against a name *string*. Because the comparison types never match, "organizations added," "contacts added," "meetings completed," and "follow-ups completed" will read as 0 for every single rep. Only Communications (a string-typed column) and Opportunities (which also OR's in `owner_id`) compute correctly. **This directly feeds the brand-new executive-report feature and the CEO-facing Analytics page** — the generated executive reports currently understate nearly every team member's activity to zero in those four categories.
- **[MEDIUM] N+1 query patterns** in the analytics and admin team views will slow down as the organization count grows.

---

7. Integrations

Integration	Status	Notes
Gmail (OAuth, send, open/click tracking, reply sync, bounce detection)	Implemented	Real two-way flow; reply-sync and bounce detection work as designed
LinkedIn automation	Partial by design	A separate in-house Chrome extension drives LinkedIn's own UI directly via browser automation — not an API integration, no reply ingestion, and the extension's own README documents ToS/ban risk. This is distinct from, and was never part of, the previously-excluded "LinkedIn automation" scope from the IHT port plan.
WhatsApp	Partial by design	Same pattern as LinkedIn: send-only, no LLM involvement, same ToS risk profile
Google Sheets	Implemented (with dead code)	Real two-way sync and a working "Sync Now" UI button; `leadTracker.ts:43` `syncSheet` is dead/duplicate code that should be removed

Integration	Status	Notes
Webhooks (inbound + outbound)	Partial	Backend fully implemented; **no frontend UI exists for managing webhook subscriptions** — unchanged since the 2026-10-28 audit
Discovery / lead research (Tavily, Serper, ScrapeGraph, LLM qualification)	Implemented, user-triggered only	Real external calls confirmed, but entirely manual — no scheduled automation runs this

Findings:

- **[HIGH] Campaign sequences never auto-advance.** `delay_days` and `next_action_due` are set correctly on each sequence step, but no scheduled job exists to send a step once it becomes due — sequences are effectively manual despite looking automated in the data model.
- **[HIGH] This worktree's** `backend/.env` **has empty values for every LLM and search API key** (TAVILY, SERPER, GROQ, GEMINI, NVIDIA, GOOGLE_SHEETS) — this contradicts an earlier direct check in this session that found GROQ_API_KEY and GEMINI_API_KEY set. This needs reconciling: most likely a different `.env` file was checked, or the worktree's copy has since diverged from the main checkout's. **Flag before relying on any LLM/search feature in this specific worktree.**
- **[MEDIUM] No time-based "no reply after N days" follow-up trigger exists** — follow-ups must be created manually even when a contact has gone silent.
- **[MEDIUM] Reply classification is pure keyword matching**, not LLM-based, despite its own docstring describing it as an "intelligent agent."
- **[LOW] Scheduled jobs are registered at module-import time rather than in the FastAPI lifespan handler** — under a hypothetical multi-worker deployment this would risk duplicate job execution. (Single-worker is currently enforced/documented — see §9 — so this is latent, not active.)

---

8. Automation (Scheduler)

Corrected from the 2026-10-28 audit: there are **6** scheduled APScheduler jobs, not 5 (a `task_reminder_sweep` job was added), and **6** `notify()` call sites, not 5. All 6 jobs were confirmed registered and running. No job silently swallows exceptions in a way that was found to crash the scheduler loop.

---

9. Security

Status of prior HIGH findings:

Finding	Status
JWT stored in `localStorage`	**Still open** — `AuthContext.tsx` reads/writes the token in localStorage; XSS would allow token theft

Finding	Status
In-memory, per-process rate limiter	**Partially addressed** — state is unchanged (still an in-memory dict), but a module docstring now explicitly documents the single-worker constraint and forbids `--workers > 1` without a shared store. Note: it keys on the raw bearer token, so a client can rotate tokens to reset its own rate limit.
Duplicate `JWT_SECRET_KEY/TOKEN_ENCRYPTION_KEY` entries in `.env`	**Fixed** — exactly one of each now
No Alembic	**Still open**
No CI	**Still open**
~50% systemic pytest error rate	**Still unverified** — no dedicated full-suite re-run was performed this pass; see §10

New findings this pass:

- **[MEDIUM]** `POST /api/geocoding/geocode` **has no authentication** and proxies directly to Nominatim — open to abuse/rate-limit exhaustion by anyone who finds the endpoint.
- **[MEDIUM] The Gmail click-tracking redirect (**`/api/gmail/track/click/{id}`**) is an open redirect** — it follows any `http(s)` URL unsigned, and because tracking IDs are sequential, open/click counts can be forged by enumeration. Should sign `(campaign_contact_id, url)`.
- **[MEDIUM] Any BD user can create an outbound webhook subscription to an arbitrary URL** — this is a realistic data-exfiltration path and should be admin-only.
- **[MEDIUM] Audit coverage is sparse and inconsistent**: contacts, meetings, follow-ups, communications, and showcase mutations write no audit record at all; opportunity update/stage-change/owner-change/delete are unaudited (only the opportunity→partnership conversion is logged); one audit call (`CREATE_TASK`) logs before `commit()`, so it stores `resource_id=None`.
- **[LOW] No server-side logout / token revocation** (see §5, Finding 7) — compounds the localStorage exposure.

---

10. Testing

Backend test suite (`test_report_agent.py` + `test_copilot.py` targeted run this session): **40 passed, 2 failed.** Both failures (`test_4_copilot_meetings_and_prep_briefing`, `test_11_copilot_context_resolution_with_active_entity`) are `IndexError: list index out of range` on an empty org search, reproducing only in full-suite ordered runs — consistent with test-isolation/ordering fragility, not a defect introduced by this branch. This is a narrow, 2-file observation and does **not** confirm or refute the prior audit's "~50% systemic pytest error rate across the 603-test, 41-file suite" — that claim still requires a dedicated full-suite run to verify or close out.

New test files were added on `main` since the last audit (`test_notifications.py`, `test_webhooks.py`, `test_knowledge_base.py`, `test_backup_service.py`, `test_signals.py`, totaling ~625 lines) — their pass/fail status was not independently verified in this pass.

---

11. Delta vs. 2026-10-28 Audit — Action Item Tracker

#	Prior action item	Status
1	Debug ~50% systemic pytest error rate	**Still open** — not re-measured with a full-suite run
2	Add Alembic	**Still open** — no `alembic/` directory, no `alembic.ini`, nothing in requirements.txt
3	Rate-limiter memory/single-worker risk	**Fixed (documented)** — docstring now states the constraint explicitly
4	Duplicate `.env` keys	**Fixed**
5	CI workflow	**Still open** — no `.github/` directory
6	Nightly backup job gap	**Still open, now worse** — latest backup file is ~3.5 weeks stale; likely cause is a WAL-mode mtime check that never sees the main DB file change
7	Tests for notifications/webhooks/KB/backups/signals	**Fixed (files added)** — pass/fail not independently re-verified
8	Untrack generated artifacts from git	**Fixed in git** — untracked local copies still exist on disk, harmless
9	Delete stale root-level `.db` files	**Fixed**
10	`NodeJS.Timeout` TypeScript error	**Fixed**
11	Webhook subscription frontend UI	**Still open** — no webhook UI anywhere in the frontend
12	Retire stale IHT port plan file	**Fixed**
13	JWT storage hardening decision	**Still open**
14	Agency-agents install decision	**Still open (presumed, unchanged)**

Tally: 7 fixed, 7 still open. Of the original 6 HIGH-severity items, 2 are closed and 4 remain open.

---

12. Consolidated Cross-Cutting Findings (highest impact first)

- **Connections board data corruption** (§4, Finding 1) — opening a page actively damages pipeline data. Highest-impact single bug in this audit.
- **Team-performance metrics silently wrong** (§6, Finding 1) — feeds both the CEO Analytics page and the newly-shipped executive report feature; will produce misleading numbers in a leadership-facing deliverable right now.

- **Role-gate gaps let HEALTHCARE/USER accounts reach BD-only data via the API** (§5, Finding 1) — a real, not theoretical, data-access violation.
- **"Success" messages that don't match reality**: `send_email` claims "dispatched" without sending; `update_organization_status` always errors; campaign sequences look automated but never advance on their own.
- **Security exposure**: open redirect in click-tracking, unauthenticated geocoding proxy, user-creatable outbound webhooks to arbitrary URLs, JWT in localStorage with no revocation path.
- **This worktree's `.env` key state is inconsistent with earlier findings this session** — verify before trusting any LLM/search-dependent feature here specifically.
- **Backups have been silently stale for ~3.5 weeks** — a real data-loss exposure if the live DB were to fail today.

---

13. Prioritized Action List

P0 — Fix before next use of the affected feature:

- Fix the Connections/Kanban stage vocabulary mismatch and disable (or correctly scope) `auto_sync_pipeline` until the 10-stage vs. 7-stage conflict is resolved.
- Fix `report_agent_service.py`'s `created_by` type mismatch (Integer vs. string) — this is actively corrupting the executive report and Analytics numbers today.
- Add `require_business_development` to Customers/Demos/Proposals/Negotiations/Tasks/Calls/Daily-Priorities/Integrations-ingest routers.
- Fix `GET /api/admin/users/{id}` (`Meeting.organizer_id` → `owner_id`).
- Reconcile the `.env` key-presence discrepancy for this worktree before using any LLM/search feature here.

P1 — High-value, low-risk fixes:

- Make `send_email`'s reply accurately describe that it logs rather than sends, or wire it to an actual send path.
- Give `update_organization_status` a real handler or remove it from the LLM tool schema.
- Add a scheduled job to advance due campaign-sequence steps.
- Fix the relationship-timeline 500 (cast `created_by`/`performed_by` to str before serializing).
- Sign the Gmail click-tracking redirect; add auth to the geocoding proxy; make outbound webhook subscription creation admin-only.
- Investigate and fix the stale-backup root cause (WAL mtime check).

P2 — Correctness/consistency cleanup:

- Unify the three activity-log mechanisms or document why they're intentionally separate.
- Remove the duplicate call-logging and opportunity→partnership conversion code paths.
- Patch the dead Gemini model IDs in `llm_router.py:58-59` (pending user decision, not yet applied).
- Fix the frontend's hardcoded `organization_id: 1` across the 5 affected views; add org pickers.
- Fix mobile notification dropdown positioning; wire Calendar event clicks; add Calls to the nav.
- Decide and document the ADMIN/DEV gate-overlap question (§5, Finding 4).

P3 — Longer-term infrastructure (carried from 2026-10-28, still unaddressed):

- Introduce Alembic for schema migrations.

- Add a CI workflow.
- Run a dedicated full-suite pytest pass to confirm or close the ~50% error-rate claim.
- Decide on JWT storage hardening (httpOnly cookie vs. continued localStorage).
- Build a webhook-subscription management UI.

This report was compiled from six parallel read-only audits conducted on 2026-10-31 against the `worktree-report-agent` branch. No source files, database rows, or secret values were modified or exposed in the course of this audit.